

Exercise 1 — Git: solve the Markdown exercise on a branch and merge it back

What you will do

The goal is to solve **Exercise 1 (Markdown)** in a git tracked notebook folder. solve it **inside your newly acquired git repository**,

The workflow is the one real teams use:

1. Copy the Whole Folder Exercise_unit_1 in your repository and make a first commit on `main` ;
2. create a **branch** for your solution;
3. **solve the Markdown exercise on that branch**, committing after every task;
4. **merge** the branch back into `main` .

Before you start (from the platform)

- **This instruction sheet.**
- **A prepared, empty repository.** Open its folder in VS Code (`File` → `Open Folder...`). If you only have an empty folder and no repository yet: open the **Source Control** panel (`Ctrl` + `Shift` + `G`) and click **Initialize Repository**.
- **The Git Graph extension.** Extensions panel (`Ctrl` + `Shift` + `X`) → search **Git Graph** (publisher *mhutchie*) → **Install**. A **Git Graph** button then appears in the Source Control title bar and in the blue status bar.
- **Your git identity** (only once per computer, if you have never done it). Open a terminal one single time and run:

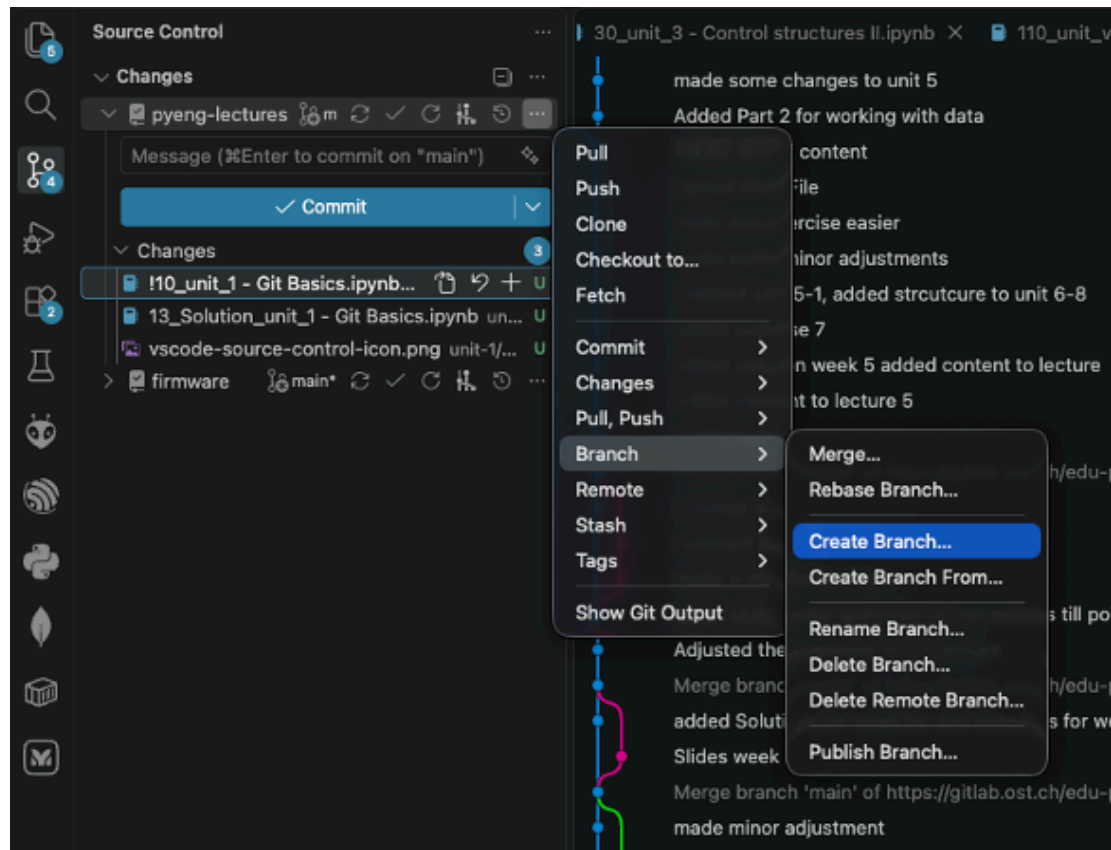
```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```
- Or if you want to use a different git account for a specific repository

```
git config --local user.name "Your Name"
git config --local user.email "you@example.com"
```

The three places you click in VS Code

Where	What you do there
Source Control panel (<code>Ctrl</code> + <code>Shift</code> + <code>G</code>)	stage files (+), write a commit message, click ✓ Commit
Branch button Next to Project 3 dots ... above Commit button	create a branch, switch branch
Git Graph window (button in the status bar / Source Control title bar) looks like a git graph	see the history; right-click a branch label to Merge or Delete it

The Source Control panel is the icon marked below in the activity bar on the left:



!

Everything below is done with these three. Whenever a step says *commit*, it means:
Source Control panel → + to stage → type the message → ✓ **Commit**.

Step 1 — Put the exercise into your repository

1. Copy the Folder **Exercise_unit_1** into your repository folder.
2. It now shows up under **Changes** in the Source Control panel. Commit it by first staging it for this you need to add a message **Add Exercise 1 (unsolved)** , click ✓ **Commit**. the changes are now staged, the button will change to **Sync** by clicking Sync the changes will be uploaded to git
3. Open **Git Graph**. You see **one commit on main** .
4. Open <https://gitlab.ost.ch/dashboard/projects> you see the files are now uploaded to your repository **Paste a screenshot of Git Graph here (one commit on main)**:

Step 2 — Create your solution branch

This step is not necessary for git to work you can always just sync your changes to main, but what if you want to develop something like a feature waiting till it's finished and testet? impossible that destroys the whole idea of agile softwaredevelopment...

for this you can work with branches, branches diverge from main you can develop independently from main and when you think everything is ready for deployment you can put all changes back to the main branch

1. Click the branch name `main` in the blue status bar (bottom-left).
2. Choose **Create new branch...**, type `solution`, confirm.
3. The status bar now shows `solution` — you are on the new branch. `main` stays where it is.

Step 3 — Solve the Markdown tasks on the branch, one commit per task

Open `10_unit_1 - Markdown.ipynb` and work through tasks **1.1 – 1.7**.

After each task, commit (Source Control → + → message → ✓ **Commit**). Use a clear message each time, for example:

- Solve 1.1 headers
- Solve 1.2 text formatting
- Solve 1.3 table
- Solve 1.4 formula
- Solve 1.5 links and images
- Solve 1.6 lists
- Solve 1.7 quote

Notes:

Aim for **at least four commits** on this branch, so the history is worth looking at in Git Graph.

Step 4 — See how the branch has diverged

1. Open **Git Graph**. The `solution` branch is now several commits ahead of `main`.
2. Click the branch name in the status bar and switch to `main`. Open the notebook — your answers are **gone** (they live safely on `solution`).
3. Switch back to `solution` — your answers are back.

In one sentence: what did the branch protect `main` from while you were solving?

Step 5 — Make `main` move too

So that the merge in the next step becomes a **real merge** (not just a shortcut), give `main` its own commit first:

1. Switch to `main` (status bar). double click on branch in git graph

2. Create a file **README.md** with your name and one line, e.g. *"My solutions to Exercise 1"*.
3. **Commit** it (**Add README**).

Now **both** branches have commits the other one does not have — open **Git Graph** and you will see the line **fork into two**.

Step 6 — Merge the solution back into **main**

1. Make sure the status bar shows **main** (you merge *into* the branch you are on).
2. In **Git Graph**, right-click the **solution** label → **Merge into current branch...**
3. Keep the suggested commit message, confirm.
4. Git creates a **merge commit** and in Git Graph the two lines **join** back into one.
5. Optional: right-click the **solution** label again → **Delete branch...** — the work is now in **main**.

If you had **skipped Step 6**, **main** would not have moved and git would do a **fast-forward** instead: no merge commit, the graph stays a straight line. Worth trying once to see the difference.

Paste a screenshot of Git Graph here (the fork and the merge commit that joins it):

Appendix — the same clicks as terminal commands (reference only)

You did everything with the GUI. For reference, these are the commands the GUI ran for you:

GUI action	Command it runs
Initialize Repository	<code>git init</code>
+ on a file	<code>git add <file></code>
message + ✓ Commit	<code>git commit -m "message"</code>
Create new branch...	<code>git switch -c <name></code>
pick a branch in the status bar	<code>git switch <name></code>
Git Graph → Merge into current branch...	<code>git merge <name></code>
Git Graph → Delete branch...	<code>git branch -d <name></code>
Push	<code>git push</code>
the Git Graph window	<code>git log --oneline --graph --all</code>